

Alex le CyberHero

Manuel d'apprentissage

Comprendre la cybersécurité en mots simples

Préparé pour Alexandre

Avant l'entrée en Master Cybersécurité

8 chapitres pour débiter de zéro et arriver prêt.

Bienvenue, Alexandre

Avant de commencer, respire un grand coup. Tu n'as **rien** besoin de savoir pour lire ce manuel. On part vraiment de zéro. Chaque mot compliqué sera expliqué avec des exemples de la vie de tous les jours.

Ce manuel est ton **livre de cours**. Il explique le *pourquoi* et le *comment* des choses, lentement et simplement. À côté, sur le dépôt GitHub, tu as un **cahier d'exercices** (un fichier par semaine) avec les manipulations à faire sur ton ordinateur. Les deux vont ensemble : tu lis un chapitre ici, puis tu pratiques dans le cahier.

Comment lire ce manuel

- **Ne saute pas les analogies.** Quand je compare un pare-feu à un videur de boîte de nuit, ce n'est pas pour faire joli : c'est pour que l'idée reste collée dans ta tête.
- **Lis à voix haute les phrases que tu ne comprends pas.** Souvent, ça suffit à débloquent.
- **Tu as le droit de ne pas tout retenir du premier coup.** La cybersécurité, c'est comme la cuisine : on comprend vraiment en pratiquant, pas en lisant la recette dix fois.
- Les encadrés comme celui ci-dessous sont des **points à retenir absolument**.

À retenir : ce manuel se lit dans l'ordre. Chaque chapitre suppose que tu as compris le précédent. Prends ton temps.

C'est quoi, la cybersécurité ?

Imagine une maison. Tu as une porte d'entrée, des fenêtres, peut-être un coffre-fort dans la chambre. La **cybersécurité**, c'est exactement le même métier, mais pour les ordinateurs et les données.

Il y a deux grandes familles de métiers :

- L'**attaquant** (on dit *red team*, l'équipe rouge) : son travail, c'est d'essayer d'entrer dans la maison comme le ferait un cambrioleur, **mais avec la permission du propriétaire**, pour trouver les points faibles avant les vrais méchants. On appelle ça un *hacker éthique* ou un *pentester* (de l'anglais *penetration tester*, testeur d'intrusion).
- Le **défenseur** (on dit *blue team*, l'équipe bleue) : son travail, c'est de poser des serrures, des alarmes, des caméras, et de surveiller. Si quelqu'un entre, il doit le voir, le bloquer, et réparer.

Un bon professionnel comprend **les deux côtés**. C'est pour ça que dans ce programme, tu vas d'abord apprendre à attaquer (pour comprendre comment les méchants pensent), puis à défendre (pour les arrêter).

La règle la plus importante de tout le manuel

On va te donner des outils puissants. Avec ces outils, on peut faire de très bonnes choses... et de très mauvaises. Alors retiens ceci, c'est non négociable :

À retenir : tu n'essaies JAMAIS d'entrer dans un ordinateur, un site ou un réseau qui ne t'appartient pas, sauf si tu as une autorisation écrite. En France, entrer sans permission dans un système informatique est un délit (article 323-1 du Code pénal), même si tu ne casses rien. Ton terrain de jeu, ce sont tes propres machines virtuelles et les sites faits exprès pour s'entraîner.

Pense à un médecin qui apprend l'anatomie : il apprend où couper pour soigner, pas pour blesser. C'est pareil ici. Ton savoir sert à **protéger**.

Le plan des 8 semaines

Voici le chemin qu'on va suivre, étape par étape. Chaque chapitre de ce manuel correspond à une semaine.

- **Semaine 1 — Le terrain de jeu.** On installe un « ordinateur dans l'ordinateur » (une machine virtuelle) et on apprend à parler à la machine avec du texte (le terminal).
- **Semaine 2 — Le réseau.** Comment les ordinateurs se parlent entre eux : adresses, chemins, langages communs.
- **Semaine 3 — Observer le réseau.** On apprend à « écouter » ce qui circule et à explorer une machine à distance.
- **Semaine 4 — Attaquer un site web.** Les failles les plus courantes des sites, et comment elles marchent.
- **Semaine 5 — Entrer dans une machine.** Comment une faille permet de prendre le contrôle d'un ordinateur.
- **Semaine 6 — Se défendre.** Fermer les portes, poser des serrures solides.
- **Semaine 7 — Surveiller.** Lire les traces, installer des alarmes, repérer une attaque.
- **Semaine 8 — Gérer une crise.** Simuler une vraie attaque du début à la fin et la résoudre comme un pro.

Allez, on y va. Tourne la page.

Chapitre 1 — On installe notre terrain de jeu

Dans ce chapitre, tu vas comprendre trois choses : ce qu'est une *machine virtuelle*, pourquoi on utilise *Linux*, et comment parler à un ordinateur en écrivant du texte dans un *terminal*. À la fin, tu auras un véritable laboratoire de hacker sur ton ordinateur, sans aucun risque pour tes fichiers personnels.

1.1 La machine virtuelle : un ordinateur dans ton ordinateur

Le problème qu'on veut résoudre

Pour apprendre la sécurité, tu vas faire des choses « dangereuses » : lancer des programmes d'attaque, casser des systèmes, tester des virus. Si tu fais ça directement sur ton ordinateur de tous les jours, tu risques de tout casser : tes photos, tes documents, ton système.

La solution s'appelle la **machine virtuelle**, qu'on abrège **VM** (de l'anglais *Virtual Machine*).

L'analogie du simulateur de vol

Un pilote d'avion ne s'entraîne pas à atterrir en tempête avec de vrais passagers à bord. Il utilise un **simulateur** : un faux cockpit où il peut tout rater, s'écraser dix fois, sans aucun danger. Le lendemain, le simulateur est comme neuf.

Une machine virtuelle, c'est exactement ça : un **faux ordinateur** qui vit à l'intérieur de ton vrai ordinateur. C'est un programme qui fait semblant d'être un PC complet, avec son propre disque dur, sa propre mémoire, son propre système. Tu peux tout casser dedans : tu fermes la fenêtre, tu redémarres, et tout est propre.

Comment c'est possible ?

Ton ordinateur est puissant. Il peut « prêter » une partie de sa puissance (par exemple 4 Go de mémoire et 2 cœurs de processeur) à un programme spécial appelé **hyperviseur**. Cet hyperviseur crée un espace isolé et y fait tourner un autre système d'exploitation, comme si c'était une vraie machine.

L'hyperviseur qu'on va utiliser s'appelle **VirtualBox**. Il est gratuit et fonctionne sur Windows, Mac et Linux.

À retenir : une machine virtuelle est un ordinateur complet simulé par un logiciel. Ce qui se passe dedans reste dedans. C'est ton bac à sable : tu peux tout y casser sans danger.

Le super-pouvoir : les instantanés (snapshots)

VirtualBox sait prendre une **photo** de l'état complet de ta machine virtuelle à un instant donné. On appelle ça un *snapshot* (instantané).

Imagine un point de sauvegarde dans un jeu vidéo. Avant un passage difficile, tu sauvegardes. Si tu meurs, tu recharges la sauvegarde. Ici, c'est pareil : avant de tester quelque chose de risqué, tu prends un snapshot. Si ça tourne mal, tu reviens à l'instantané en quelques secondes, comme si rien ne s'était passé.

1.2 Linux : le système des informaticiens

Trois systèmes d'exploitation, trois philosophies

Un **système d'exploitation** (en anglais *OS*, pour *Operating System*) est le programme principal qui fait tourner ton ordinateur : Windows, macOS et Linux sont des systèmes d'exploitation. C'est le chef d'orchestre qui gère l'écran, le clavier, les fichiers, les autres programmes.

Tu connais sûrement Windows. En sécurité, on utilise surtout **Linux**. Pourquoi ?

- **Linux est ouvert (open source).** Cela veut dire que n'importe qui peut lire le code qui le compose. Rien n'est caché. Pour des gens dont le métier est la confiance et la sécurité, c'est précieux.
- **Linux est gratuit et léger.** Il tourne très bien dans une machine virtuelle, même avec peu de mémoire.
- **Linux est le pays des outils de sécurité.** La plupart des outils de hacking sont faits d'abord pour Linux.
- **Linux s'automatise facilement.** On peut écrire de petits programmes (des *scripts*) qui font le travail répétitif à ta place.

Kali : la trousse à outils du pentester

Linux existe en plusieurs versions, qu'on appelle des **distributions** (ou *distros*). C'est comme une même voiture vendue en plusieurs finitions.

- **Kali Linux** est une distribution faite *spécialement* pour la sécurité offensive. Elle arrive avec plus de 600 outils déjà installés. C'est la trousse à outils du hacker éthique. C'est celle qu'on installe cette semaine.
- **Ubuntu** est une distribution généraliste, simple, qu'on utilise souvent comme *serveur* à défendre. On l'ajoutera plus tard, dans la partie défense.

Comment Linux range ses fichiers

Sous Windows, tu as l'habitude des lecteurs **C:** et **D:**. Sous Linux, c'est différent : **tout part d'un seul point**, qu'on appelle la **racine** et qu'on note avec une simple barre oblique : **/**.

À partir de cette racine, tout est rangé dans des dossiers. Voici les plus importants :

- **/bin** : les commandes de base du système (les outils livrés avec).
- **/etc** : tous les fichiers de **réglage** (la configuration). C'est le cerveau des

réglages.

- `/home` : les dossiers personnels des utilisateurs. Ton territoire à toi.
- `/var` : les fichiers qui grossissent avec le temps, surtout les **journaux** (*logs*). On y reviendra beaucoup en semaine 7.
- `/root` : le dossier personnel du super-chef du système (l'utilisateur *root*).

À retenir : sous Linux, tout descend d'un seul dossier racine noté /. Les réglages sont dans /etc, tes fichiers dans /home, et les traces du système dans /var.

1.3 Le terminal : parler à la machine avec des mots

Cliquer, c'est bien ; écrire, c'est puissant

Tu as l'habitude de cliquer sur des icônes avec ta souris. C'est confortable, mais limité : tu ne peux faire que ce que les boutons proposent.

Le **terminal** (cette fenêtre noire avec du texte qui fait peur au début) est une autre façon de commander l'ordinateur : tu **écris** ce que tu veux, et la machine le fait. C'est plus puissant parce que tu peux tout faire, combiner des actions, et les automatiser.

L'analogie du restaurant

Cliquer sur des boutons, c'est comme choisir un menu en pointant les photos du doigt : pratique, mais tu es limité aux photos affichées. Le terminal, c'est comme **parler directement au chef** et lui décrire exactement le plat que tu veux. Au début c'est intimidant, mais tu obtiens précisément ce que tu demandes.

Chaque ligne que tu écris s'appelle une **commande**. Tu tapes la commande, tu appuies sur Entrée, la machine répond.

Les commandes de survie

Voici les commandes que tu utiliseras tous les jours. Apprends-les comme tu as appris l'alphabet.

Commande	Ce qu'elle fait	Exemple
<code>pwd</code>	Affiche où tu te trouves (le dossier courant)	<code>pwd</code>
<code>ls</code>	Liste ce qu'il y a dans le dossier	<code>ls -la</code>
<code>cd</code>	Change de dossier (se déplacer)	<code>cd /home</code>
<code>mkdir</code>	Crée un nouveau dossier	<code>mkdir test</code>
<code>cat</code>	Affiche le contenu d'un fichier	<code>cat notes.txt</code>
<code>nano</code>	Ouvre un éditeur pour écrire dans un fichier	<code>nano notes.txt</code>
<code>rm</code>	Supprime un fichier (attention, définitif)	<code>rm vieux.txt</code>

cp	Copie un fichier	cp a.txt b.txt
mv	Déplace ou renomme un fichier	mv a.txt b.txt
sudo	Exécute une commande en mode super-chef	sudo apt update
man	Affiche le mode d'emploi d'une commande	man ls

Petite explication de deux d'entre elles, parce qu'elles reviennent sans arrêt :

- **ls -la** : le **ls** liste les fichiers. Le **-la** ajoute des **options** : **l** pour afficher les détails (taille, date, droits) et **a** pour montrer *aussi* les fichiers cachés. En Linux, un fichier dont le nom commence par un point (comme **.bashrc**) est caché par défaut.
- **sudo** veut dire « *super user do* », c'est-à-dire « fais ceci en tant que super-utilisateur ». Certaines actions (installer un programme, modifier le système) demandent les droits du chef. **sudo** te les donne le temps d'une commande. C'est comme passer une carte d'accès « administrateur » devant un lecteur.

Le danger à connaître par cœur

Une commande peut effacer des fichiers. La plus dangereuse de toutes est celle-ci :

```
rm -rf /
```

Décortiquons-la pour comprendre pourquoi elle est terrible. **rm** supprime. **-r** veut dire « et tout ce qu'il y a à l'intérieur, en descendant dans tous les sous-dossiers ». **-f** veut dire « force, ne me pose aucune question ». Et **/** est la racine, c'est-à-dire **tout le système**. Cette commande dit donc : « supprime absolument tout, sans rien me demander ».

*À retenir : ne tape jamais **rm -rf /** « pour voir ». Même dans une machine virtuelle, prends l'habitude de te méfier de **rm**. Un mauvais réflexe pris dans le bac à sable peut un jour coûter cher sur une vraie machine.*

1.4 Les permissions : qui a le droit de faire quoi

Pourquoi des permissions ?

Sur un ordinateur, plusieurs personnes (ou plusieurs programmes) peuvent vivre ensemble. On ne veut pas que n'importe qui puisse lire le journal intime de n'importe qui, ni modifier les réglages du système. Linux gère donc des **permissions** : pour chaque fichier, il note qui a le droit de quoi.

Les trois droits et les trois groupes

Il y a **trois droits** possibles sur un fichier :

- **lire** (en anglais *read*, abrégé **r**) : voir le contenu.

- **écrire** (*write*, abrégé **w**) : modifier le contenu.
- **exécuter** (*execute*, abrégé **x**) : lancer le fichier s'il s'agit d'un programme.

Et ces droits sont donnés à **trois catégories** de personnes :

- le **propriétaire** du fichier (celui qui l'a créé),
- le **groupe** (une équipe d'utilisateurs),
- les **autres** (tout le reste du monde).

Les chiffres magiques (644, 755...)

Tu verras souvent des permissions écrites avec trois chiffres, comme **644** ou **755**. Voici l'astuce pour les lire. On donne une valeur à chaque droit : lire vaut **4**, écrire vaut **2**, exécuter vaut **1**. On additionne pour obtenir le chiffre d'une catégorie.

- Lire + écrire + exécuter = 4 + 2 + 1 = **7** (tous les droits).
- Lire + écrire = 4 + 2 = **6**.
- Lire + exécuter = 4 + 1 = **5**.
- Lire seul = **4**.

On met ensuite un chiffre par catégorie, dans l'ordre propriétaire / groupe / autres.

Permission	Propriétaire	Groupe	Autres	Quand l'utiliser
644	lire + écrire (6)	lire (4)	lire (4)	Un fichier de texte normal
755	tous (7)	lire + exécuter (5)	lire + exécuter (5)	Un programme que tout le monde peut lancer
600	lire + écrire (6)	rien (0)	rien (0)	Un fichier secret (mot de passe, clé)

Pour changer les permissions, on utilise la commande **chmod** (de *change mode*). Par exemple, pour rendre un script lançable : **chmod +x mon_script.sh**. Le **+x** ajoute le droit d'exécution.

À retenir : 4 = lire, 2 = écrire, 1 = exécuter. On additionne, et on écrit un chiffre par catégorie (propriétaire, groupe, autres). chmod +x rend un fichier exécutable.

1.5 Ton premier script

Un **script** est un fichier qui contient une liste de commandes. Au lieu de taper les commandes une par une, tu les écris dans un fichier et tu lances le fichier : toutes les commandes s'exécutent d'un coup. C'est ta première automatisation.

Un script commence toujours par une ligne spéciale appelée *shebang* :

```
#!/bin/bash
```


Cette ligne dit à l'ordinateur : « le programme qui doit lire ce fichier s'appelle `bash` » (`bash` est le nom du terminal Linux le plus courant). Ensuite, tu écris tes commandes, une par ligne. Pour pouvoir lancer le script, tu le rends exécutable avec `chmod +x`, puis tu l'appelles avec `./mon_script.sh` (le `./` veut dire « le fichier qui est ici, dans le dossier courant »).

Tu mettras tout ça en pratique dans le cahier d'exercices de la semaine 1.

Ce qu'il faut retenir de ce chapitre

- Une **machine virtuelle** est un ordinateur simulé : ton bac à sable sans risque.
- Les **snapshots** sont des points de sauvegarde : prends-en avant chaque expérience risquée.
- **Linux** est le système des pros de la sécurité ; **Kali** est sa version « trousse à outils ».
- Le **terminal** te laisse commander la machine en écrivant ; c'est plus puissant que la souris.
- Les **permissions** (644, 755, 600) décident qui peut lire, écrire ou exécuter.
- Un **script** automatise une série de commandes.

Passe maintenant au cahier d'exercices de la semaine 1 pour installer ta VM et fabriquer ton premier script. Quand tu reviens, on attaque le réseau.

Chapitre 2 — Le réseau, comme un pro

Tout, en cybersécurité, tourne autour du réseau. Une attaque, c'est presque toujours une machine qui parle à une autre machine. Si tu comprends *comment* les ordinateurs se parlent, tu comprends où on peut écouter, mentir, ou bloquer. Ce chapitre t'explique ça en partant de zéro.

2.1 C'est quoi, un réseau ?

Un **réseau**, c'est simplement plusieurs ordinateurs reliés entre eux pour échanger des informations. Ta box internet à la maison relie ton téléphone, ta télé et ton PC : c'est un petit réseau. Internet, c'est le plus grand réseau du monde, fait de millions de petits réseaux reliés entre eux.

Quand deux ordinateurs s'échangent une information, ils ne s'envoient pas un gros bloc d'un coup. Ils découpent le message en petits morceaux appelés **paquets**, un peu comme on enverrait un long courrier en plusieurs enveloppes numérotées. Chaque paquet voyage de son côté, et la machine d'arrivée les remet dans l'ordre. Cette idée de « paquet » reviendra sans arrêt.

2.2 Le modèle en couches : un message qui voyage par étapes

L'analogie de la lettre postale

Imagine que tu écris une lettre à un ami à l'étranger. Plusieurs personnes interviennent sans que tu les voies :

- **Toi**, tu écris le contenu de la lettre.
- Tu la mets dans une **enveloppe** avec l'adresse.
- **La Poste** de ton pays décide par quel chemin l'envoyer.
- **Les avions et les camions** la transportent physiquement.
- À l'arrivée, tout se fait dans l'autre sens : transport, tri, enveloppe ouverte, lettre lue.

Chaque acteur a un rôle précis et ne s'occupe pas du travail des autres. Le facteur n'a pas besoin de lire ta lettre pour la livrer. Internet fonctionne exactement comme ça : par **couches**, chacune avec son rôle, empilées les unes sur les autres.

Le modèle OSI : 7 couches

Les informaticiens ont défini un modèle de référence à **7 couches**, appelé le **modèle OSI**. Tu n'as pas besoin de le réciter par cœur tout de suite, mais tu dois comprendre l'idée : un message descend les 7 couches chez l'expéditeur, voyage, puis remonte les 7 couches chez le destinataire.

Couche	Nom	Son rôle, en simple	Exemple
--------	-----	---------------------	---------

7	Application	Là où vit le programme que tu utilises	Ton navigateur web
6	Présentation	Chiffre et met en forme les données	Le cadenas HTTPS
5	Session	Ouvre et ferme la conversation	—
4	Transport	Vérifie que tout arrive en entier	TCP, UDP
3	Réseau	Trouve le chemin entre les machines	Les adresses IP
2	Liaison	Parle aux machines juste à côté	Le câble, le Wi-Fi
1	Physique	Le signal réel : électricité, ondes	La prise réseau

À retenir : un message traverse des couches, de la plus proche de l'humain (couche 7, l'application) à la plus proche du matériel (couche 1, le câble). Chaque couche a un seul rôle.

Pourquoi ça compte en sécurité

Quand une attaque arrive, savoir à quelle couche elle frappe te dit comment te défendre. Un faux site web qui te vole ton mot de passe, c'est la couche 7. Une inondation de fausses connexions pour saturer un serveur, c'est plutôt la couche 4. Quelqu'un qui se fait passer pour ta box sur le Wi-Fi, c'est la couche 2. Le modèle OSI est ta **carte mentale** : il te dit où regarder.

Le modèle TCP/IP : la version courte

En vrai, dans la pratique, on simplifie les 7 couches OSI en **4 couches**, qu'on appelle le modèle **TCP/IP** (c'est le modèle réellement utilisé sur Internet).

Couche TCP/IP	Correspond à	Exemples
Application	OSI 7, 6, 5	Web, mail, DNS
Transport	OSI 4	TCP, UDP
Internet	OSI 3	IP
Accès réseau	OSI 2, 1	Wi-Fi, câble

2.3 TCP et UDP : deux façons d'envoyer

À la couche transport, il existe deux grandes méthodes pour envoyer des paquets.

- **TCP** est la méthode **fiable**. Avant d'envoyer, les deux machines se mettent d'accord. Pour chaque paquet reçu, la machine d'arrivée renvoie un petit « bien reçu ». Si un paquet se perd, on le renvoie. C'est comme une lettre recommandée avec accusé de réception : lent, mais sûr. On l'utilise pour le web, les mails, les transferts de fichiers.

- **UDP** est la méthode **rapide**. On envoie les paquets sans vérifier qu'ils arrivent. C'est comme jeter des cartes postales dans une boîte : certaines peuvent se perdre, mais c'est rapide et léger. On l'utilise pour la vidéo en direct, les jeux en ligne, le DNS.

La poignée de main TCP

Pour ouvrir une connexion TCP, les deux machines font un petit rituel en trois temps, appelé *3-way handshake* (poignée de main en trois étapes) :

1. La machine A dit : « Salut, je veux te parler » (message **SYN**).
2. La machine B répond : « OK, je t'entends, et toi tu m'entends ? » (message **SYN-ACK**).
3. La machine A confirme : « Oui, c'est parti » (message **ACK**).

Retiens cette séquence **SYN, SYN-ACK, ACK** : on la retrouvera en analysant du trafic, et certaines attaques consistent justement à abuser de cette poignée de main.

À retenir : TCP est fiable et confirme chaque paquet ; UDP est rapide mais sans garantie. Toute connexion TCP commence par la poignée de main SYN, SYN-ACK, ACK.

2.4 L'adresse IP : l'adresse postale d'une machine

Pour qu'une machine en trouve une autre, il lui faut une **adresse**. C'est l'**adresse IP** (IP veut dire *Internet Protocol*).

Une adresse IP classique (version 4, dite IPv4) ressemble à ça :

192.168.1.10

C'est quatre nombres entre 0 et 255, séparés par des points. Chaque machine sur un réseau a la sienne, comme chaque maison a une adresse postale unique dans sa rue.

Adresses privées et adresses publiques

Il y a deux mondes :

- Les **adresses privées** sont utilisées à l'intérieur d'un réseau local (ta maison, une entreprise). Elles ne sont pas visibles depuis Internet. On les reconnaît : elles commencent souvent par 192.168., 10., ou 172.16 à 172.31. C'est comme le numéro d'un bureau à l'intérieur d'une entreprise : utile dedans, inconnu dehors.
- Les **adresses publiques** sont visibles sur Internet. C'est l'adresse que ta box présente au monde entier.

Deux adresses spéciales à connaître :

- 127.0.0.1 : c'est **toi-même**, ta propre machine. On l'appelle l'adresse de *loopback* (boucle locale). Quand un programme veut se parler à lui-même, il utilise cette adresse.
- 255.255.255.255 : c'est l'adresse de **diffusion** (*broadcast*), qui parle à tout le

monde sur le réseau en même temps.

2.5 Le masque de sous-réseau : qui est mon voisin ?

Le problème

Quand ta machine veut envoyer un paquet, elle doit savoir une chose : la destination est-elle **dans mon réseau** (mon voisin direct), ou **ailleurs** (il faut passer par la box) ? Pour répondre, elle utilise le **masque de sous-réseau**.

L'analogie de l'immeuble

Pense à une adresse comme « Immeuble 192.168.1, Appartement 10 ». Le masque, c'est ce qui sépare la partie « immeuble » de la partie « appartement ». Si deux personnes sont dans le **même immeuble**, elles peuvent se parler directement dans le couloir. Si elles sont dans des immeubles différents, il faut passer par la rue (la box, qu'on appelle la *passerelle* ou *gateway*).

Un masque très courant est 255.255.255.0, qu'on écrit aussi /24. Le /24 veut dire : « les 24 premiers bits désignent l'immeuble (le réseau), le reste désigne l'appartement (la machine) ».

Un petit calcul simple

Avec un /24, sur l'adresse 192.168.1.10 :

- L'**adresse du réseau** (le nom de l'immeuble) est 192.168.1.0.
- Les **machines possibles** vont de 192.168.1.1 à 192.168.1.254.
- L'**adresse de diffusion** (parler à tout l'immeuble) est 192.168.1.255.

Pour calculer combien de machines un réseau peut contenir : on compte les bits laissés aux machines, et on fait « 2 puissance ce nombre, moins 2 ». Pour un /24, il reste 8 bits, donc 2 puissance 8 = 256, moins 2 = **254 machines**. Pourquoi moins 2 ? Parce qu'on enlève l'adresse du réseau et l'adresse de diffusion, qui ne sont pas attribuables à une vraie machine.

Notation	Masque	Machines possibles
/24	255.255.255.0	254
/25	255.255.255.128	126
/26	255.255.255.192	62

À retenir : le masque sépare la partie « réseau » de la partie « machine » d'une adresse IP. Il sert à savoir si une machine est ton voisin direct ou s'il faut passer par la passerelle.

2.6 Les protocoles et les ports : des langages et des portes

Les protocoles, ce sont des langages

Une fois que deux machines se trouvent, encore faut-il qu'elles se comprennent. Pour ça, elles parlent un langage commun appelé **protocole** : un ensemble de règles précises. Le web parle le protocole **HTTP**, les mails parlent **SMTP**, etc.

Les ports, ce sont des portes numérotées

Une seule machine peut faire plusieurs choses en même temps : héberger un site web, recevoir des mails, accepter des connexions à distance. Pour ne pas tout mélanger, chaque service écoute derrière une **porte numérotée** appelée **port**.

Reprends l'image de l'immeuble : l'adresse IP, c'est l'immeuble ; le port, c'est le numéro de l'appartement où vit un service précis. Le service web habite traditionnellement à l'appartement (port) 80, le service web sécurisé au 443, et ainsi de suite.

Voici les ports à connaître absolument, comme tu connais les numéros d'urgence :

Protocole	Port	À quoi il sert	Sécurisé ?
HTTP	80	Naviguer sur le web	Non, en clair
HTTPS	443	Web avec cadenas	Oui, chiffré
SSH	22	Se connecter à distance	Oui
FTP	21	Transférer des fichiers	Non, vieux
DNS	53	Traduire un nom en adresse IP	À surveiller
SMTP	25	Envoyer des mails	À surveiller
RDP	3389	Bureau à distance Windows	À surveiller

À retenir : l'adresse IP désigne la machine, le port désigne le service sur cette machine. Quand tu entends « le port 22 est ouvert », pense aussitôt « SSH, on peut s'y connecter à distance ».

2.7 Le DNS : l'annuaire d'Internet

Tu ne tapes jamais [142.250.179.78](#) pour aller sur Google : tu tapes [google.com](#). Mais les machines, elles, ne travaillent qu'avec des adresses IP. Il faut donc un **traducteur** entre les noms (faciles pour les humains) et les adresses (utiles pour les machines). Ce traducteur, c'est le **DNS** (*Domain Name System*).

Le DNS est exactement comme l'annuaire téléphonique d'autrefois : tu connais le nom de quelqu'un, l'annuaire te donne son numéro. Tu connais [google.com](#), le DNS te donne l'adresse IP correspondante.

Quand tu tapes une adresse dans ton navigateur, ta machine demande discrètement au DNS « quelle est l'adresse de ce nom ? », reçoit la réponse, puis se connecte. Tout ça en une fraction de seconde, sans que tu le voies.

Comme le DNS est au cœur de tout, c'est une cible favorite des attaquants : s'ils arrivent à te mentir sur l'adresse d'un site (« non non, la vraie banque, c'est cette adresse-là »), ils peuvent t'envoyer sur un faux site. On en reparlera dans la partie défense.

Ce qu'il faut retenir de ce chapitre

- Un réseau découpe les messages en **paquets** qui voyagent puis sont rassemblés.
- Le modèle en **couches** (OSI à 7 couches, TCP/IP à 4) répartit les rôles : application, transport, réseau, accès.
- **TCP** est fiable, **UDP** est rapide ; TCP commence par la poignée de main SYN, SYN-ACK, ACK.
- L'**adresse IP** identifie une machine ; le **masque** dit qui est dans le même réseau.
- Le **port** identifie un service ; apprend les ports courants (22, 80, 443, 53).
- Le **DNS** traduit les noms en adresses, comme un annuaire.

Au prochain chapitre, on passe à la pratique de l'observation : on va « écouter » le réseau et explorer une machine à distance.

Chapitre 3 — Observer le réseau

Maintenant que tu sais comment les machines se parlent, on va apprendre à **regarder** ce qui circule et à **explorer** une machine à distance. Ce sont les deux yeux du professionnel de la sécurité : écouter le trafic (avec Wireshark) et cartographier une cible (avec Nmap).

3.1 Wireshark : le microscope du réseau

Voir l'invisible

À chaque instant, des centaines de paquets entrent et sortent de ta machine. Tu ne les vois pas. **Wireshark** est un outil qui les **capture** et te les montre un par un, en détail. C'est un microscope : il rend visible ce qui d'habitude file trop vite pour l'œil.

Concrètement, tu choisis quelle carte réseau écouter (par exemple `eth0`, ta connexion principale), tu lances la capture, et Wireshark affiche en direct chaque paquet : qui l'envoie, à qui, avec quel protocole, et ce qu'il contient.

Trier dans le déluge : les filtres

Le problème, c'est qu'il y a *trop* de paquets. Pour t'y retrouver, Wireshark a des **filtres** : tu écris une petite condition, et il ne te montre que ce qui t'intéresse. Quelques filtres utiles :

Filtre	Ce qu'il montre
http	Seulement le trafic web non chiffré
dns	Seulement les demandes à l'annuaire DNS
ip.addr == 192.168.1.10	Seulement les paquets d'une machine précise
tcp.port == 443	Seulement le trafic web sécurisé

Il y a aussi une fonction très pratique appelée **Follow TCP Stream** (suivre le flux TCP) : tu cliques droit sur un paquet, et Wireshark reconstitue la **conversation complète** entre les deux machines, comme si tu lisais tout le dialogue d'un coup.

Le moment où tu comprends pourquoi HTTPS existe

Voici l'expérience qui marque tout le monde. Si tu captures une connexion à un site en **HTTP** (le web non sécurisé, port 80) et que tu y tapes un mot de passe, tu retrouves ce mot de passe **écrit en clair** dans les paquets de Wireshark. N'importe qui qui écoute le réseau peut le lire.

Si tu refais la même chose sur un site en **HTTPS** (le web sécurisé, port 443), tu ne vois plus rien : tout est **chiffré**, transformé en charabia illisible.

C'est ça, le fameux petit cadenas de ton navigateur : la garantie que ce que tu envoies est brouillé et que personne ne peut le lire en route.

À retenir : en HTTP, tout circule en clair et peut être lu par qui écoute. En HTTPS, tout est chiffré. Wireshark te le montre de tes propres yeux : c'est la meilleure leçon de sécurité qui soit.

3.2 Nmap : explorer une machine à distance

La reconnaissance, première étape de tout

Avant d'attaquer (ou de défendre) une machine, il faut la **connaître** : quelle est son adresse, quels services elle fait tourner, quelles portes sont ouvertes. Cette phase d'exploration s'appelle la **reconnaissance**. C'est comme un cambrioleur honnête qui ferait d'abord le tour de la maison pour noter les portes et fenêtres — sauf qu'ici, c'est avec autorisation, pour réparer les faiblesses.

L'outil roi de la reconnaissance s'appelle **Nmap** (*Network Mapper*, le cartographe de réseau).

Ce que Nmap sait faire

Nmap envoie de petits paquets à une machine et déduit beaucoup de choses des réponses :

- Quelles **machines** sont allumées sur un réseau.
- Quels **ports** sont ouverts (donc quels services tournent).
- Quelle **version** de logiciel se cache derrière chaque service.
- Parfois même quel **système d'exploitation** tourne.

Quelques exemples de commandes :

Commande	Ce qu'elle fait
<code>nmap 192.168.1.10</code>	Scan de base des ports d'une machine
<code>nmap -sV 192.168.1.10</code>	Trouve la version des services
<code>nmap -p- 192.168.1.10</code>	Teste les 65535 ports
<code>nmap -A 192.168.1.10</code>	Scan complet : OS, versions, scripts
<code>nmap -sn 192.168.1.0/24</code>	Liste les machines allumées du réseau

Comment Nmap devine l'état d'un port

C'est plus simple qu'il n'y paraît. Nmap frappe à la porte (envoie un paquet) et écoute la réponse :

- Si la machine répond « entre » (un paquet SYN-ACK), le port est **ouvert** (*open*) : un service écoute derrière.
- Si la machine répond « va-t'en » (un paquet RST), le port est **fermé** (*closed*) : personne n'écoute.
- Si la machine ne répond **rien du tout**, le port est **filtré** (*filtered*) : un pare-feu bloque silencieusement. C'est souvent le signe d'une machine bien protégée.

Retiens bien ces trois mots anglais : **open, closed, filtered**. Ce sont les réponses que Nmap t'affichera.

À retenir : Nmap est ton explorateur. Il révèle les machines vivantes, les ports ouverts et les services. Tu ne scannes JAMAIS une machine sans permission : utilise tes propres VM, ou la cible publique d'entraînement scanme.nmap.org.

3.3 Plonger dans le DNS

On a vu que le DNS traduit les noms en adresses. En tant que futur professionnel, tu dois savoir l'**interroger** toi-même, car il regorge d'informations sur une cible.

Le DNS ne stocke pas qu'une simple correspondance nom vers adresse. Il garde plusieurs **types d'enregistrements** :

- **A** : le nom vers une adresse IPv4 (le plus courant).
- **AAAA** : le nom vers une adresse IPv6 (la nouvelle génération d'adresses).
- **MX** : les serveurs qui reçoivent les **mails** de ce domaine.
- **NS** : les serveurs DNS responsables du domaine.
- **TXT** : des informations diverses, souvent liées à la sécurité des mails.

L'outil principal pour interroger le DNS s'appelle **dig**. Par exemple, **dig google.com** te donne l'adresse, et **dig MX gmail.com** te dit quels serveurs reçoivent les mails de Gmail. Un autre outil, **whois**, te dit **qui possède** un nom de domaine et depuis quand.

Pourquoi un attaquant s'intéresse au DNS ? Parce qu'en interrogeant l'annuaire d'une entreprise, il peut découvrir ses serveurs, ses sous-domaines, son fournisseur de mail — bref, dessiner la carte de sa cible sans même la toucher directement. C'est de la reconnaissance « discrète ».

Ce qu'il faut retenir de ce chapitre

- **Wireshark** capture et affiche les paquets : c'est ton microscope. Les **filtres** t'aident à trier.
- En **HTTP** tout est en clair, en **HTTPS** tout est chiffré : Wireshark te le prouve.
- **Nmap** explore une cible : machines vivantes, ports ouverts, services, versions.
- Un port est **ouvert, fermé** ou **filtré** selon la réponse de la machine.
- Le **DNS** s'interroge avec **dig** et **whois** ; il révèle beaucoup sur une cible.
- On n'observe et on ne scanne **que** ce qu'on a le droit de toucher.

Tu as maintenant les yeux. Au prochain chapitre, on apprend à se servir de ce qu'on voit : on attaque un site web (dans un laboratoire, bien sûr).

Chapitre 4 — Attaquer un site web (en laboratoire)

Les sites web sont partout, et ce sont les cibles les plus fréquentes. Dans ce chapitre, tu vas comprendre comment fonctionnent les failles web les plus courantes : pourquoi elles existent, comment elles marchent, et comment on les corrige. Tout se fait sur des sites **volontairement troués**, installés chez toi pour l'entraînement.

À retenir : tout ce chapitre se pratique sur des applications faites exprès pour ça (DVWA, OWASP Juice Shop), installées dans ta machine virtuelle. On n'attaque jamais un vrai site. Jamais.

4.1 Comment marche un site web, en deux minutes

Pour attaquer (ou défendre) un site, il faut comprendre le va-et-vient de base.

- Ton **navigateur** (Chrome, Firefox) est le **client**. Il demande des pages.
- Le **serveur** est l'ordinateur, quelque part, qui héberge le site et répond.
- Souvent, derrière le serveur, il y a une **base de données** : un grand classeur où sont rangés les comptes, les mots de passe, les commandes, les messages.

Quand tu te connectes à un site, voici ce qui se passe en gros : ton navigateur envoie ton identifiant et ton mot de passe au serveur ; le serveur va vérifier dans la base de données si ça correspond ; si oui, il te laisse entrer. Les failles web viennent presque toujours d'un serveur qui fait **trop confiance** à ce que le navigateur lui envoie.

4.2 L'OWASP Top 10 : la liste des dangers

Une organisation appelée **OWASP** (un groupe mondial d'experts de la sécurité web) publie régulièrement la liste des **10 risques les plus graves** pour les sites web. C'est la référence que tout professionnel connaît. On l'appelle l'**OWASP Top 10**.

Tu n'as pas besoin de tout mémoriser aujourd'hui, mais retiens que les deux failles les plus emblématiques, celles qu'on étudie ici, en font partie : l'**injection** (notamment l'injection SQL) et le **XSS**. Voyons-les en détail.

4.3 L'injection SQL : faire dire à la base ce qu'on veut

D'abord, c'est quoi SQL ?

La base de données comprend un langage appelé **SQL**. C'est avec ce langage qu'on lui pose des questions. Par exemple, quand tu te connectes, le serveur pose à la base une question qui ressemble à : « donne-moi l'utilisateur dont le nom est *Alexandre* et le mot de passe est *xyz* ». En SQL, ça s'écrit à peu près :

```
SELECT * FROM users WHERE name = 'Alexandre' AND password = 'xyz';
```

Le problème : mélanger les instructions et les données

Voici la faille. Souvent, le serveur **fabrique** cette question en collant directement ce que tu as tapé dans le champ. Si tu tapes ton nom, il colle ton nom. Mais que se passe-t-il si, au lieu d'un nom, tu tapes un bout de langage SQL ?

C'est ça, l'**injection SQL** : tu glisses du code SQL dans un champ de formulaire pour modifier la question que le serveur pose à la base.

L'exemple classique

Imagine que dans le champ « nom », au lieu d'écrire ton nom, tu écris :

```
' OR '1'='1
```

Le serveur, qui colle bêtement, fabrique alors cette question :

```
SELECT * FROM users WHERE name = '' OR '1'='1' AND password = '...';
```

Regarde le morceau `OR '1'='1'`. Il veut dire « OU BIEN si 1 égale 1 ». Or, 1 égale toujours 1 ! La condition est donc **toujours vraie**, et la base répond « oui, c'est bon, entre ». Tu viens de te connecter **sans connaître le mot de passe**. C'est exactement comme un videur à qui tu dirais « laisse-moi entrer SI je suis invité OU BIEN si le ciel est bleu » — et le ciel est toujours un peu bleu.

Ce qu'on peut en faire, et comment on s'en protège

Avec des injections plus avancées, on peut faire bien pire : lire toute la base, extraire les mots de passe de tous les utilisateurs, etc. Un outil appelé `sqlmap` automatise ce travail pour tester si un site est vulnérable.

La **défense** est simple à dire : il ne faut **jamais coller** ce que l'utilisateur tape directement dans une question SQL. On utilise des **requêtes préparées**, une technique où les données restent toujours séparées des instructions. La base sait alors que `' OR '1'='1` est juste un texte bizarre tapé comme nom, pas une instruction.

À retenir : l'injection SQL exploite un serveur qui mélange les données de l'utilisateur avec ses instructions à la base. La parade, c'est de toujours séparer les deux (requêtes préparées).

4.4 Le XSS : injecter du code dans la page

L'idée

XSS veut dire *Cross-Site Scripting*. Le principe : au lieu de viser la base de données, tu injectes du **code JavaScript** (le langage qui anime les pages web) dans une page. Ce code s'exécutera ensuite dans le navigateur des **autres visiteurs**.

Comment ça arrive

Imagine un site avec une zone de commentaires. Tu écris un commentaire, il s'affiche

pour tout le monde. Maintenant, que se passe-t-il si, au lieu d'un texte normal, tu écris un morceau de JavaScript dans le commentaire ? Si le site est mal protégé, il va **afficher** ton commentaire en l'exécutant comme du code. Chaque visiteur qui ouvre la page exécute ton script sans le savoir.

Un test inoffensif classique consiste à faire apparaître une petite fenêtre d'alerte :

```
<script>alert('XSS par Alexandre')</script>
```

Si une boîte de dialogue s'affiche, la faille est confirmée.

Les trois familles de XSS

- **Reflected** (réfléchi) : le code est renvoyé tout de suite, généralement via un lien piégé qu'on t'envoie.
- **Stored** (stocké) : le code est **enregistré** sur le site (dans un commentaire, un profil) et frappe **tous** les visiteurs. C'est le plus dangereux.
- **DOM-based** : la faille est dans le JavaScript déjà présent sur la page.

Pourquoi c'est grave

Avec un XSS, un attaquant peut voler les **cookies de session** des visiteurs. Un cookie de session, c'est le petit jeton que le site te donne après ta connexion pour se souvenir que c'est toi. Si l'attaquant le vole, il peut se faire passer pour toi sans connaître ton mot de passe. Il peut aussi rediriger les visiteurs, enregistrer ce qu'ils tapent, ou défigurer la page.

La **défense** consiste à toujours « échapper » ce que les utilisateurs écrivent, c'est-à-dire l'afficher comme du texte inoffensif et jamais comme du code.

À retenir : le XSS injecte du JavaScript qui s'exécute chez les visiteurs. Le XSS stocké est le pire car il touche tout le monde. La parade : ne jamais afficher l'entrée d'un utilisateur comme du code.

4.5 Burp Suite : voir et modifier les requêtes

Quand tu navigues, ton navigateur envoie des **requêtes** au serveur. Normalement, tu ne les vois pas. **Burp Suite** est un outil qui se place **entre** ton navigateur et le site (on appelle ça un *proxy*, un intermédiaire) et te montre chaque requête. Mieux : tu peux les **modifier** avant qu'elles partent.

L'analogie : Burp, c'est comme intercepter une lettre à la poste, l'ouvrir, changer un mot, la refermer et la laisser repartir. Pour tester un site, c'est extrêmement puissant : tu peux changer un prix, un identifiant, un paramètre, et voir comment le serveur réagit.

Le module le plus utile pour débiter s'appelle **Repeater** (le répéteur) : il te permet de renvoyer une même requête autant de fois que tu veux, en la modifiant à chaque essai, pour observer les réponses.

Ce qu'il faut retenir de ce chapitre

- Les failles web viennent d'un serveur qui fait **trop confiance** à ce que le navigateur envoie.
- L'**OWASP Top 10** liste les 10 risques web majeurs ; l'injection et le XSS en font partie.
- L'**injection SQL** glisse du code SQL dans un formulaire pour tromper la base de données.
- Le **XSS** injecte du JavaScript qui s'exécute chez les visiteurs ; le stocké est le plus grave.
- **Burp Suite** intercepte et modifie les requêtes : un outil d'analyse indispensable.
- Les défenses existent et sont simples à nommer : séparer données et instructions, et échapper les sorties.

Tu sais maintenant attaquer un site. Au prochain chapitre, on monte d'un cran : on prend le contrôle d'une machine entière.

Chapitre 5 — Entrer dans une machine

Jusqu'ici, tu as observé et tu as joué avec des sites web. Maintenant, on s'attaque à une **machine entière** : comment une faiblesse permet d'en prendre le contrôle à distance. C'est le cœur du métier de pentester. Et, comme toujours, ça se passe dans ton laboratoire isolé.

5.1 Comment se déroule une attaque : la « kill chain »

Une vraie attaque ne se fait pas au hasard. Elle suit des **étapes**, qu'on appelle la *kill chain* (la « chaîne d'attaque »). Les comprendre te servira aussi bien pour attaquer que pour défendre, car chaque étape est une occasion de se faire repérer.

1. **Reconnaissance** : récolter des informations sur la cible (le chapitre 3, avec Nmap).
2. **Préparation de l'arme** : fabriquer l'outil qui exploitera la faille.
3. **Livraison** : faire arriver cet outil jusqu'à la cible (un mail piégé, une connexion réseau).
4. **Exploitation** : déclencher la faille.
5. **Installation** : poser une porte dérobée pour pouvoir revenir.
6. **Contrôle à distance** : piloter la machine depuis chez soi.
7. **Objectif final** : voler des données, saboter, etc.

À retenir : une attaque est une suite d'étapes, pas un coup de chance. Côté défense, chaque étape est une chance de détecter et d'arrêter l'attaquant avant qu'il aille plus loin.

5.2 Le vocabulaire de l'exploitation

Avant d'utiliser l'outil, fixons trois mots simples.

- Une **vulnérabilité** (ou *faille*) est une faiblesse dans un programme : un bug, une erreur de conception, une vieille version non corrigée.
- Un **exploit** est un bout de code qui **profite** d'une vulnérabilité précise pour faire faire à la machine quelque chose qu'elle ne devrait pas.
- Un **payload** (la « charge utile ») est ce qui s'exécute **une fois qu'on est entré**. Par exemple : ouvrir un accès qui te donne la main sur la machine.

L'analogie : la vulnérabilité, c'est une fenêtre mal fermée. L'exploit, c'est le geste qui l'ouvre. Le payload, c'est ce que tu fais une fois à l'intérieur (allumer la lumière, ouvrir la porte de derrière...).

5.3 Metasploit : la boîte à outils de l'exploitation

Metasploit est le logiciel le plus connu pour exploiter des failles. Imagine une

immense boîte à outils qui contient des milliers d'exploits déjà prêts, classés et faciles à lancer. Tu choisis ta cible, tu choisis l'exploit, tu choisis le payload, et tu lances.

On l'utilise dans une console appelée `msfconsole`. Le déroulé typique ressemble à ceci :

```
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
show options
set RHOSTS 192.168.56.101
run
```

Décortiquons :

- `search vsftpd` cherche un exploit visant un service appelé vsftpd.
- `use ...` sélectionne l'exploit trouvé.
- `show options` affiche les réglages à remplir.
- `set RHOSTS ...` indique l'adresse de la cible (*RHOST* = *remote host*, la machine distante).
- `run` lance l'attaque.

Si tout se passe bien, tu obtiens un **shell** : une invite de commande **sur la machine cible**, comme si tu étais physiquement devant elle. À partir de là, tu peux taper des commandes sur sa machine.

5.4 Le reverse shell : faire revenir la cible vers toi

Le problème des pare-feu

Tu voudrais ouvrir une connexion vers la machine cible pour la commander. Mais les pare-feu bloquent en général les connexions qui **arrivent** de l'extérieur. En revanche, ils laissent souvent **sortir** les connexions. Comment contourner ça ?

L'astuce du rappel téléphonique

On utilise un **reverse shell** (un shell « inversé »). Au lieu que toi tu appelles la cible, c'est la **cible qui t'appelle**.

L'analogie : tu veux joindre quelqu'un dont le standard bloque les appels entrants. Tu lui fais passer un mot : « rappelle ce numéro ». C'est elle qui compose, donc l'appel **sort** de chez elle et passe le standard sans problème. Une fois la ligne ouverte, vous discutez normalement.

Concrètement : tu prépares ta machine pour qu'elle **écoute** (un *listener*), puis tu fais en sorte que la cible lance une connexion vers toi. Quand elle « rappelle », tu obtiens le contrôle. L'inverse, où c'est toi qui te connectes à un port ouvert sur la cible, s'appelle un *bind shell* ; il est plus facilement bloqué par les pare-feu.

À retenir : un reverse shell fait que la cible se connecte à l'attaquant, ce qui contourne les pare-feu qui bloquent les connexions entrantes mais laissent sortir.

5.5 Meterpreter et la post-exploitation

Une fois entré, certains payloads te donnent un outil très puissant appelé **Meterpreter**. Ce n'est pas un simple shell : c'est un véritable poste de pilotage de la machine cible. Avec lui, tu peux :

- Voir les informations du système ([sysinfo](#)).
- Vérifier quel utilisateur tu es ([getuid](#)).
- Récupérer les mots de passe stockés ([hashdump](#)).
- Prendre une capture d'écran ([screenshot](#)).

Tout ce qu'on fait **après** être entré s'appelle la **post-exploitation** : explorer la machine, récupérer des informations, et parfois rebondir vers d'autres machines du réseau (on appelle ça le *pivot* : utiliser une machine compromise comme tremplin vers une autre).

5.6 Le laboratoire d'attaque : un réseau isolé

Pour t'entraîner, tu utiliseras une machine cible volontairement pleine de failles, appelée **Metasploitable**. C'est une passoire pédagogique : parfaite pour apprendre, catastrophique à exposer sur Internet.

C'est pourquoi tu placeras Kali (l'attaquant) et Metasploitable (la cible) dans un **réseau privé isolé**, sans accès à Internet. Dans VirtualBox, ce mode s'appelle **Host-Only** (réseau hôte uniquement). Les deux machines se voient l'une l'autre, mais le monde extérieur ne les voit pas, et elles ne sortent pas. C'est ta salle d'entraînement fermée à clé.

À retenir : on n'entraîne l'exploitation que sur des cibles prévues pour ça (Metasploitable), dans un réseau isolé sans Internet. Une cible volontairement vulnérable ne doit jamais être accessible de l'extérieur.

Ce qu'il faut retenir de ce chapitre

- Une attaque suit des **étapes** (la kill chain) ; chacune est détectable.
- **Vulnérabilité** = la faiblesse ; **exploit** = ce qui en profite ; **payload** = ce qui s'exécute ensuite.
- **Metasploit** est la grande boîte à outils d'exploitation, pilotée dans [msfconsole](#).
- Un **reverse shell** fait revenir la cible vers toi pour contourner les pare-feu.
- **Meterpreter** est un poste de pilotage avancé ; la **post-exploitation**, c'est tout ce qu'on fait une fois entré.
- On s'entraîne sur **Metasploitable** dans un réseau **isolé**.

Tu sais maintenant comment on entre dans une machine. À partir du prochain chapitre, on change de camp : on apprend à **empêcher** tout ça.

Chapitre 6 — Se défendre comme un boss

Changement de camp. Jusqu'ici tu jouais l'attaquant, pour comprendre comment pensent les méchants. Maintenant, tu deviens le **défenseur** (la *blue team*). Ton but : prendre une machine et la rendre **difficile à attaquer**. On appelle ça le **durcissement** (en anglais *hardening*).

6.1 L'état d'esprit du défenseur

Réduire la surface d'attaque

Imagine ta machine comme un château. Chaque porte, chaque fenêtre, chaque service qui tourne est une **entrée possible** pour un attaquant. L'ensemble de ces entrées s'appelle la **surface d'attaque**. Tout le travail de défense consiste à **réduire** cette surface : moins il y a de portes ouvertes, moins il y a de risques.

Les quatre grands principes

- **Le moindre privilège.** Chaque utilisateur, chaque programme ne reçoit que les droits **strictement nécessaires**, rien de plus. Le stagiaire n'a pas les clés du coffre-fort. Si un compte est piraté, les dégâts restent limités.
- **Désactiver l'inutile.** Un service que tu n'utilises pas est une porte ouverte pour rien. On l'éteint.
- **Mettre à jour.** La grande majorité des attaques exploitent des failles **déjà connues et déjà corrigées**. Mettre à jour, c'est fermer ces portes. C'est l'action de sécurité la plus rentable qui existe.
- **La défense en profondeur.** On ne met pas une seule serrure, mais plusieurs barrières successives : pare-feu, mots de passe forts, surveillance. Si une barrière tombe, les autres tiennent encore.

À retenir : durcir une machine, c'est réduire le nombre de portes ouvertes et empiler les protections. Mettre à jour est l'action la plus rentable de toutes.

6.2 Le pare-feu : le vider de la machine

Le **pare-feu** (en anglais *firewall*) est le vider à l'entrée de ta machine. Il regarde chaque paquet qui veut entrer ou sortir et décide, selon des **règles**, s'il le laisse passer ou non.

La bonne stratégie est claire et tient en une phrase : **tout bloquer par défaut, puis n'ouvrir que le strict nécessaire**. C'est la règle de la « liste blanche » : on interdit tout, et on autorise seulement ce qu'on connaît. C'est bien plus sûr que d'essayer d'interdire au cas par cas les choses dangereuses (on en oublierait toujours une).

Sous Ubuntu, l'outil simple pour gérer le pare-feu s'appelle **ufw** (*Uncomplicated Firewall*, le pare-feu sans complications). Quelques commandes typiques :

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw enable
```

En français : on bloque tout ce qui entre, on autorise tout ce qui sort, on ouvre quand même le port 22 (pour pouvoir se connecter à distance en SSH), puis on active le pare-feu. Après ça, si tu refais un scan Nmap depuis Kali, tu verras la différence : presque tous les ports apparaissent maintenant comme **filtrés**. La surface d'attaque a fondu.

6.3 Sécuriser l'accès à distance (SSH)

SSH est le service qui permet de se connecter à une machine à distance, en ligne de commande. Il est extrêmement utile... et donc extrêmement visé. Sur Internet, des robots tentent en permanence de deviner les mots de passe SSH de toutes les machines qu'ils trouvent. Il faut donc le blinder.

Voici les réglages essentiels (ils se trouvent dans un fichier de configuration appelé `sshd_config`) :

- **Interdire la connexion directe du super-utilisateur.** Le compte *root* est le plus puissant ; on interdit de s'y connecter directement (`PermitRootLogin no`). Un attaquant doit ainsi deviner deux choses au lieu d'une.
- **Se connecter par clé plutôt que par mot de passe.** Plutôt qu'un mot de passe (qu'on peut deviner), on utilise une **paire de clés** : une clé privée que tu gardes secrète sur ta machine, et une clé publique que tu déposes sur le serveur. Pour entrer, il faut posséder la clé privée. C'est comme une serrure ultra-spéciale dont une seule clé existe au monde, dans ta poche. On désactive alors le mot de passe (`PasswordAuthentication no`).

L'analogie de la clé : un mot de passe, c'est un code que quelqu'un peut deviner ou voir par-dessus ton épaule. Une clé privée, c'est un objet physique unique : sans l'avoir, impossible d'entrer, même en connaissant tout le reste.

À retenir : SSH doit être blindé. Interdis la connexion root directe, et préfère l'authentification par clé au mot de passe. Une clé bien protégée vaut mille mots de passe.

6.4 fail2ban : bannir automatiquement les intrus

Même avec SSH bien réglé, des robots vont essayer encore et encore de se connecter. **fail2ban** est un petit gardien automatique : il **surveille les journaux** de la machine et, dès qu'une même adresse IP échoue trop de fois à se connecter (par exemple 3 fois), il la **bannit** : il la bloque au niveau du pare-feu pendant un certain temps.

C'est exactement le videur qui dit : « tu t'es trompé de mot de passe trois fois,

dehors, et tu reviens dans une heure ». L'attaquant automatique, qui voulait essayer des milliers de mots de passe, se fait jeter après trois essais. Son attaque devient inutile.

Tu pourras le tester toi-même : depuis Kali, tu lanceras un outil qui tente plein de mots de passe ([hydra](#)), et tu verras ton adresse se faire bannir au bout de quelques essais. Sensation très satisfaisante.

6.5 Vérifier son travail : l'audit

Comment savoir si ta machine est bien durcie ? On utilise un outil d'**audit** qui passe le système en revue et note ce qui va et ce qui ne va pas. Un outil gratuit et excellent s'appelle **Lynis** : il analyse des centaines de points et te donne un score de durcissement, avec des conseils d'amélioration.

Il existe aussi des **guides officiels** de référence : les **CIS Benchmarks** (internationaux) et, en France, les **guides de durcissement de l'ANSSI** (l'agence nationale de cybersécurité). Ce sont des documents qui listent, point par point, comment configurer un système proprement. En connaître l'existence et savoir les appliquer fait partie du métier.

À retenir : on ne devine pas si une machine est sûre, on le mesure. Lynis audite ta machine ; les guides ANSSI et CIS te donnent la marche à suivre officielle.

Ce qu'il faut retenir de ce chapitre

- Défendre, c'est **réduire la surface d'attaque** : moins de portes ouvertes, plus de barrières.
- Les quatre principes : **moindre privilège, désactiver l'inutile, mettre à jour, défense en profondeur**.
- Le **pare-feu** bloque tout par défaut et n'ouvre que le nécessaire ; [ufw](#) le rend simple.
- **SSH** se blinde : pas de root direct, connexion par **clé**.
- **fail2ban** bannit automatiquement les machines qui s'acharnent.
- On **mesure** sa sécurité avec un audit (**Lynis**) et on suit les guides (**ANSSI, CIS**).

Tu sais maintenant fermer les portes. Mais que faire si quelqu'un entre quand même ? Au prochain chapitre : surveiller et détecter.

Chapitre 7 — Surveiller et détecter

Aucune défense n'est parfaite. Un attaquant déterminé finira peut-être par entrer. La question n'est donc pas seulement « comment l'empêcher ? », mais aussi « comment **le voir** quand il essaie, et **réagir** ? ». C'est le métier de la surveillance, le cœur du travail d'un analyste en sécurité.

7.1 Les journaux : la mémoire de la machine

Tout laisse une trace

Chaque action importante sur une machine est notée quelque part : une connexion, une erreur, un programme qui démarre. Ces notes s'appellent les **journaux** (en anglais *logs*). C'est la mémoire écrite du système, comme le registre d'un hôtel où l'on note chaque entrée et sortie.

Sous Linux, les journaux vivent dans le dossier `/var/log`. Quelques fichiers importants :

Fichier	Ce qu'il contient
<code>/var/log/auth.log</code>	Les connexions et tentatives de connexion
<code>/var/log/syslog</code>	Les messages généraux du système
<code>access.log</code>	Les visites d'un site web

Lire les journaux intelligemment

Un journal peut contenir des dizaines de milliers de lignes. On ne les lit pas une par une : on les **filtre**. Avec des commandes comme `grep` (chercher un mot), `awk` (extraire une colonne) et quelques autres, on peut par exemple sortir, en une seule ligne, la liste des adresses qui ont **échoué** à se connecter, classées de la plus insistante à la moins insistante.

C'est exactement ce que fait un analyste quand il enquête : il pose une question précise aux journaux, et les journaux répondent. Savoir lire les logs est une compétence de base ultra-recherchée.

À retenir : les journaux sont la mémoire de la machine. Savoir les lire et les filtrer (avec `grep`, `awk`...) est la compétence fondamentale de la détection.

7.2 Les systèmes de détection d'intrusion (IDS / IPS)

Lire les journaux à la main, c'est bien pour enquêter, mais on ne peut pas rester les yeux dessus 24 heures sur 24. On installe donc des **gardiens automatiques**.

- Un **IDS** (*Intrusion Detection System*, système de détection d'intrusion) surveille et **donne l'alerte** quand il repère quelque chose de suspect. C'est une caméra avec alarme : elle prévient, mais n'agit pas.

- Un **IPS** (*Intrusion Prevention System*, système de prévention) va plus loin : il détecte **et bloque** automatiquement. C'est la caméra qui, en plus de sonner, ferme la grille toute seule.

Ces gardiens existent en deux saveurs : ceux qui surveillent le **réseau** (le trafic qui circule, comme Suricata ou Snort) et ceux qui surveillent une **machine** précise (ses fichiers, ses journaux).

Deux façons de repérer le danger

- **Par signatures** : le gardien connaît une liste de « visages » d'attaques déjà répertoriées et sonne quand il en reconnaît une. Très efficace contre le connu, mais aveugle face à une attaque toute neuve.
- **Par anomalies** : le gardien apprend d'abord à quoi ressemble une journée **normale**, puis sonne dès que quelque chose sort de l'ordinaire. Capable de repérer l'inconnu, mais il se trompe parfois (fausses alertes).

7.3 Le SIEM : la tour de contrôle

Dans une vraie organisation, il y a des dizaines, voire des milliers de machines, chacune produisant ses journaux. Impossible de tout suivre séparément. On utilise alors un **SIEM**.

Un **SIEM** (les lettres signifient *Security Information and Event Management*) est une **tour de contrôle** qui rassemble les journaux de tout le parc au même endroit, les met en relation, et lève des alertes. Pense à la tour de contrôle d'un aéroport qui voit tous les avions sur un seul écran au lieu de surveiller chaque piste séparément.

La magie de la corrélation

La vraie force d'un SIEM, c'est la **corrélation** : relier des événements qui, séparément, sembleraient anodins.

Un exemple : un échec de connexion, ce n'est rien. Mais **200 échecs en une minute** depuis la même adresse, **suivis d'une connexion réussie**, c'est l'histoire complète d'un mot de passe finalement deviné. Le SIEM relie ces points et lève une alerte rouge, là où un humain n'aurait vu que des lignes isolées. C'est en reliant les indices qu'on découvre une attaque.

Un SIEM open source très utilisé pour apprendre s'appelle **Wazuh**. Tu en installeras un dans le cahier d'exercices.

À retenir : un SIEM centralise tous les journaux et corrèle les événements pour révéler une attaque que chaque ligne, prise seule, ne montrerait pas. C'est la tour de contrôle de la sécurité.

7.4 Le métier qui se cache derrière : analyste SOC

Tout ce que tu apprends dans ce chapitre porte un nom de métier : **analyste SOC**.

Un **SOC** (*Security Operations Center*, centre des opérations de sécurité) est l'équipe qui surveille en permanence les systèmes d'une organisation, lit les alertes du SIEM, enquête, et réagit.

C'est l'un des **premiers postes** accessibles quand on sort d'une école de cybersécurité. En maîtrisant la lecture de journaux et le fonctionnement d'un SIEM **avant même de commencer ton Master**, tu prends une vraie longueur d'avance.

Ce qu'il faut retenir de ce chapitre

- Les **journaux** sont la mémoire de la machine ; savoir les filtrer est une compétence clé.
- Un **IDS** alerte, un **IPS** alerte **et** bloque.
- On détecte **par signatures** (le connu) ou **par anomalies** (l'inhabituel).
- Un **SIEM** centralise les journaux et les **corrèle** pour révéler les attaques.
- Tout ça, c'est le métier d'**analyste SOC**, une excellente porte d'entrée dans la profession.

Tu sais maintenant fermer les portes ET repérer les intrus. Au dernier chapitre, on assemble tout : on gère une vraie crise du début à la fin.

Chapitre 8 — Gérer une crise : la réponse à incident

Voici le grand final. Tu vas assembler tout ce que tu as appris pour faire face à une vraie situation : une machine a été piratée, et c'est à toi de gérer. Ce chapitre t'explique la méthode des professionnels pour traiter un incident calmement et efficacement.

8.1 Garder la tête froide : un incident se gère par étapes

Quand une attaque est découverte, le réflexe du débutant est de paniquer : tout débrancher, tout effacer, dans le désordre. C'est une erreur. Les professionnels suivent une **méthode** en étapes, définie par un organisme de référence américain (le NIST). Retiens ces six phases :

1. **Préparation.** Avant même qu'un incident arrive, on s'équipe : outils prêts, procédures écrites, sauvegardes à jour. On ne s'organise pas pendant l'incendie, mais avant.
2. **Détection et analyse.** On repère l'incident (souvent grâce au SIEM du chapitre 7) et on mesure son ampleur : qu'est-ce qui est touché, et jusqu'où ?
3. **Confinement.** On **isole** la zone touchée pour empêcher la propagation, comme on ferme une porte coupe-feu. Par exemple : couper l'accès de l'attaquant, débrancher la machine du réseau.
4. **Éradication.** On **supprime la cause** : le programme malveillant, le compte créé par l'attaquant, la faille qu'il a utilisée.
5. **Récupération.** On remet les services en marche proprement, à partir d'un état sain, et on surveille de près pour vérifier que l'attaquant ne revient pas.
6. **Leçons apprises.** Après la crise, on fait le bilan : qu'est-ce qui a marché, qu'est-ce qu'on améliore pour la prochaine fois ?

À retenir : un incident se gère en six phases (préparation, détection, confinement, éradication, récupération, leçons apprises). On ne débranche pas tout en panique : on suit la méthode.

8.2 La règle d'or : tout documenter

Pendant un incident, **chaque action doit être notée, avec l'heure**. Pourquoi est-ce si important ?

- Pour **comprendre** ce qui s'est passé et le raconter ensuite.
- Pour ne pas se contredire ni oublier une étape.
- Parce qu'un incident peut avoir des **suites judiciaires** : les traces que tu

collectes peuvent devenir des **preuves**. Si tu les manipules n'importe comment, elles perdent leur valeur.

On parle de **chaîne de preuve** : comme dans une enquête de police, chaque indice doit être collecté, noté et conservé proprement, sinon il ne vaut plus rien au tribunal.

8.3 Mener l'enquête : les indices de compromission

Pour comprendre une intrusion, on cherche des **indices de compromission** (en anglais *IOC, Indicators of Compromise*). Ce sont les traces que l'attaquant a laissées :

- une **adresse IP** inhabituelle qui s'est connectée,
- un **compte utilisateur** qui est apparu sans qu'on l'ait créé,
- une **tâche programmée** suspecte qui se lance toute seule,
- une **clé** ajoutée pour permettre un retour discret,
- un **fichier** ou un programme étrange.

Sur une machine Linux, quelques commandes simples permettent d'enquêter : voir qui s'est connecté récemment, quels programmes tournent, quels fichiers ont été modifiés dans les dernières 24 heures, et quelles tâches automatiques sont programmées. C'est le travail du détective : relever les empreintes.

La persistance : la porte de derrière de l'attaquant

Un attaquant malin ne se contente pas d'entrer : il installe de quoi **revenir** même si on change le mot de passe. On appelle ça la **persistance**. Cela peut être un compte caché, une clé d'accès ajoutée, ou un programme qui rouvre une connexion à intervalles réguliers. Lors de l'éradication, c'est précisément ce qu'il faut traquer et supprimer, sinon l'attaquant reviendra par sa porte de derrière.

À retenir : on cherche les indices de compromission (comptes créés, tâches suspectes, connexions étranges) et on traque la persistance, ces portes de derrière que l'attaquant laisse pour revenir.

8.4 Le livrable du pro : le rapport d'incident

À la fin, un professionnel produit un **rapport d'incident**. Ce document raconte l'histoire complète et sert à la fois aux chefs, aux techniciens et, parfois, à la justice. Un bon rapport contient :

- Un **résumé pour les non-techniciens** (3 ou 4 phrases : que s'est-il passé, quel impact, est-ce réglé).
- Une **chronologie** des événements, avec les heures.
- Le **point d'entrée** de l'attaquant et les **indices** trouvés.
- Les **actions menées** pour régler le problème.
- Des **recommandations** pour que ça ne se reproduise pas.

Savoir écrire clairement est une compétence aussi importante que savoir hacker. Un

rapport confus rend inutile le meilleur travail technique. Un modèle de rapport t'attend dans les annexes du dépôt.

8.5 Ton épreuve finale

Dans le cahier d'exercices de la semaine 8, tu joueras les **deux rôles** sur ta propre machine de test :

- D'abord **l'attaquant** : tu compromets ta machine Ubuntu, tu y installes une persistance, et tu notes tout ce que tu fais (ce sera ton corrigé secret).
- Puis le **défenseur** : tu repars de zéro, tu **détectes** l'intrusion dans les journaux et le SIEM, tu **confinas**, tu **éradiques**, tu **récupères**, et tu rédiges ton **rapport**.

C'est l'exercice qui réunit tout : Linux, réseau, attaque, défense, surveillance. Si tu le réussis, tu auras fait, en miniature, le travail réel d'un professionnel de la cybersécurité.

Ce qu'il faut retenir de ce chapitre

- Un incident se gère en **six phases**, avec méthode et sans panique.
- On **documente tout, avec l'heure** : les traces sont des preuves (chaîne de preuve).
- On cherche les **indices de compromission** et on traque la **persistance**.
- On termine par un **rapport** clair, lisible par tous.
- L'épreuve finale réunit toutes tes compétences : attaque, défense, détection, réaction.

Le mot de la fin

Alexandre, si tu as lu ce manuel jusqu'ici et pratiqué les exercices, tu n'arrives pas en Master comme un débutant. Tu arrives avec un laboratoire monté, des réflexes des deux côtés de la barrière, et un vocabulaire de professionnel.

Tu n'as pas besoin d'être un génie. Juste d'être **curieux et régulier**. Une heure par jour vaut mieux qu'une nuit blanche une fois par mois.

Maintenant, démarre cette machine virtuelle. Le jeu commence pour de vrai.